

Bus Tracking Backend (Node.js + Express + EJS)

Backend + dashboard for a city transportation graduation system. Provides:

- Manager web dashboard (EJS) with session-based authentication
 - Hardware ingestion endpoints to store live bus location updates
 - Public JSON endpoints for the Flutter app (routes + active buses + nearest bus)
 - Telegram test endpoint to validate the hardware → backend → Telegram message flow
-

Current Scope

- Clean layered architecture (models, services, controllers, routes, middlewares, config)
 - Session-based manager authentication for dashboard pages
 - Mongoose schemas: Company, Route, Bus, LocationLog
 - Security middleware (Helmet)
 - Hardware ingestion: store/update bus location via `/api/hardware/location`
 - Telegram test: store/update bus location and send a message via `/api/test/telegram`
 - Public/mobile read APIs for Flutter
 - Dashboard pages for route management and bus log monitoring
-

Project Structure

- `src/config`: environment + MongoDB connection
 - `src/models`: Mongoose schemas
 - `src/services`: business logic
 - `src/controllers`: request handlers
 - `src/routes/web`: SSR dashboard/auth routes
 - `src/routes/api`: JSON API routes
 - `src/middlewares`: auth + error handling
 - `src/views`: EJS templates
-

Prerequisites

- Node.js (LTS recommended)
 - MongoDB (local or remote)
 - Telegram Bot token (only needed if you use `/api/test/telegram`)
-

Setup

1. Install dependencies

```
npm install
```

2. Configure environment variables

- Create a `.env` file based on `.env.example`
 - Required keys (see `src/config/env.js`):
 - `PORT` (default: `3000`)
 - `MONGO_URI` (default: `mongodb://127.0.0.1:27017/trackly`)
 - `SESSION_SECRET` (required for sessions; default fallback is `change-this-in-env` but you should override)
 - `TELEGRAM_BOT_TOKEN` (default: empty)
 - `TELEGRAM_DEFAULT_CHAT_ID` (default: empty)
-

Run

Development

```
npm run dev
```

Production

```
npm start
```

Server starts on `http://localhost:${PORT}`.

Dashboard (Web) Flow

1. Open `/auth/register` and register a company manager.
 2. After registration, copy and store the generated `apiSecret` (this is used by hardware requests).
 3. Login via `/auth/login`.
 4. Create/manage routes using the dashboard pages.
 5. Open `/buses` to inspect buses and their logs.
-

API Endpoints (Base: `/api`)

1) Write/Test Endpoints (hardware ingestion)

POST `/api/hardware/location`

Stores hardware location and updates the bus status/location.

Body (JSON): use the same fields as `api-testing.rest`:

- `apiSecret` (company secret)
- `deviceId` (unique device id for the bus)
- `busNumber`

- `routeId`
- `latitude`
- `longitude`
- `speed`
- `timestamp` (ISO string)

POST `/api/test/telegram`

Same storage behavior as hardware ingestion, then sends a formatted message to Telegram.

Body (JSON): same as above plus optionally:

- `chatId` (if not provided, falls back to `TELEGRAM_DEFAULT_CHAT_ID`)
-

2) Public Read Endpoints (for Flutter)

GET `/api/public/routes`

List routes. You can optionally filter by company:

- `companyId` (optional query param)

GET `/api/public/routes/:routeId/buses`

Get active buses for a given route.

GET `/api/public/routes/:routeId/nearest?latitude=<lat>&longitude=<lng>`

Get the nearest active bus for the given coordinates.

3) Dashboard Route API

POST `/api/routes`

Session-auth required (manager dashboard).

Body (JSON) (based on `api-testing.rest`):

- `routeName`
 - `coordinates` (array of [`lng`, `lat`] pairs)
-

Using `api-testing.rest`

The repo includes a ready-to-use REST Client file:

- `api-testing.rest`

It defines variables like:

- `baseUrl`
- `companyApiSecret`, `companyId`, `routeId`, `deviceId`, `chatId`

It contains example requests for:

- `/api/hardware/location`
- `/api/test/telegram`
- `/api/public/routes`
- `/api/public/routes/:routeId/buses`
- `/api/public/routes/:routeId/nearest`

Tip: update `baseUrl` and the variable values to match your deployed/local server data.

Troubleshooting

- **MongoDB connection fails:** check `MONGO_URI` in `.env`.
 - **Dashboard login/session issues:** ensure `SESSION_SECRET` is set to a strong random value.
 - **Telegram messages don't arrive:**
 - verify `TELEGRAM_BOT_TOKEN`
 - verify `TELEGRAM_DEFAULT_CHAT_ID` (or pass `chatId` to `/api/test/telegram`)
-

Update Log

- 2026-04-14
 - Refactored into clean layered architecture
 - Added session auth, Helmet, telemetry ingestion, Telegram test endpoint
 - Added public mobile-read APIs and dashboard pages
 - Added README and REST Client endpoint collection
-

Next Suggested Steps

- Add API authentication + rate limiting for public write endpoints
- Add richer company admin features (bus assignment to route, stop management)
- Add WebSocket/SSE for real-time updates in dashboard + Flutter
- Add tests (services/controllers) and request validation